

Terraform on AWS, Explained

A private network, PostgreSQL 17, a CRUD website, Keycloak and NiFi — built in stacks, explained line by line

Written in plain language, with grocery-store and school analogies, for anyone who wants to build and run a cloud platform

September 2026

Contents

1. How to Read This Book

This book teaches you to build a small, private AWS environment with Terraform, one piece at a time, and then to look after it the way a cloud team does. It is written so a curious middle schooler could follow it, but everything in it is real and used by professionals.

You will see four kinds of boxes throughout:

KEY TIP: Key tips

Green boxes: short habits that save you hours.

GOTCHA: Gotchas

Orange boxes: the mistakes almost everyone makes once. Read these twice.

ANALOGY: Analogies

Blue boxes: the same idea explained with a grocery store, a classroom, or a school campus.

Gray boxes with a blue stripe are code. Every code box is followed by a plain-language explanation of **what** it does and **why** it is there.

The project has six folders called stacks. You need the first three. Stacks 4, 5 and 6 are optional add-ons you can try later.

Stack	What it builds	Required?
01-network	A private VPC with two Availability Zones, four subnets, route table and VPC endpoints. No internet gateway.	Yes
02-database	A secret (rcadmin / changeme), two security groups, RDS PostgreSQL 17.	Yes
03-compute	IAM role, launch template with a CRUD website, Auto Scaling Group, one EC2.	Yes
04-database-restore	A second database restored from a snapshot with a brand-new password.	Optional
05-keycloak	A Keycloak login server on the existing VPC and RDS, with a sample realm named nifi.	Optional
06-nifi	Apache NiFi logging in through Keycloak with OIDC.	Optional

2. The Big Picture: A School Campus

ANALOGY: The whole project is a school campus

The **VPC** is the campus: fenced, with its own address range. This campus has **no front gate** (no internet gateway). Nobody drives in from the street, and nobody drives out.

Subnets are hallways. Two hallways are for classrooms (app subnets, where servers live) and two are for the records office (db subnets, where the database lives). Each pair is split across **two buildings** (Availability Zones) so a power cut in one building does not close the school.

The **route table** is the map on the wall. Ours only shows how to walk between hallways; there is no arrow labeled "to the street".

VPC endpoints are a few locked delivery doors in the fence, each leading straight to one specific supplier: the package warehouse (S3), the safe company (Secrets Manager) and the intercom company (Session Manager). Nothing else can use those doors.

Security groups are the hall monitors at each classroom door. They have a list: who may come in, on which port.

RDS is the records office with a librarian who guards the files. Only servers wearing the right badge may ask for records.

Secrets Manager is the principal's safe. The librarian's key is kept there, not taped under a desk.

IAM roles are staff badges. A badge says what a server is allowed to ask AWS for.

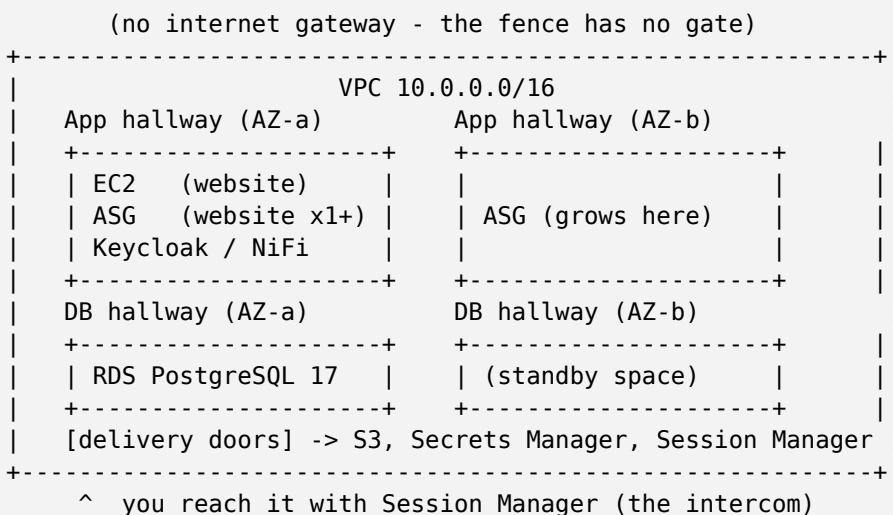
A **launch template** is a substitute-teacher packet: everything needed to set up a classroom identically every time.

The **Auto Scaling Group** is the vice-principal who makes sure there are always enough teachers in the building and calls a substitute when one goes home sick.

Session Manager is the intercom: you can talk to any classroom from the office without walking in through a gate.

Here is the campus drawn as a map:

diagram



3. Terraform in Plain Language

3.1 What Terraform is

Terraform is a program that reads text files describing what you want to exist in the cloud, compares that with what already exists, and then makes the changes. You never click through the AWS website. This idea is called **Infrastructure as Code**.

ANALOGY: A recipe versus cooking by memory

Clicking in the AWS console is like cooking by memory: it works once, but you cannot hand it to someone else, and you will not remember what you did last month. A Terraform file is a written recipe. Anyone can follow it, you can fix a typo and re-run it, and you can keep every version in a binder (Git).

3.2 The words you must know

Word	Meaning	Looks like
Provider	The plugin that knows how to talk to one cloud	<code>provider "aws" { region = "us-east-1" }</code>
Resource	One thing you want to exist	<code>resource "aws_vpc" "this" { ... }</code>
Data source	One thing that already exists and you only want to look up	<code>data "aws_vpc" "this" { ... }</code>
Variable	A knob with a default that tfvars can turn	<code>var.instance_type</code>
Local	A value you compute once and reuse	<code>local.user_data</code>
Output	A value printed when apply finishes	<code>output "rds_endpoint" { ... }</code>
Module	A folder of resources you can reuse like a LEGO brick	<code>module "vpc" { source = "../modules/vpc" }</code>
State	The file <code>terraform.tfstate</code> where Terraform remembers what it built	never edit it by hand

3.3 The shape of every resource

```
hcl
resource "TYPE" "NICKNAME" {
  setting = value
}
```

TYPE is the AWS thing (`aws_vpc`, `aws_instance`, `aws_db_instance`). NICKNAME is your name for it inside Terraform only; AWS never sees it. To use one resource inside another you write `TYPE.NICKNAME.attribute`, for example `aws_vpc.this.id`.

3.4 The workflow

1. `terraform init`: download the providers. Run once per folder, and again if you add a provider.
2. `terraform plan`: show what would change. Nothing is built. Read it.

3. `terraform apply`: build it. Type yes.
4. `terraform destroy`: remove everything this folder built.

KEY TIP: Always plan before apply

A plan that says "1 to add, 0 to change, 1 to destroy" when you expected only an add is Terraform warning you. The word destroy in a plan should always make you stop and read.

GOTCHA: The state file is the memory

If `terraform.tfstate` is deleted, Terraform forgets everything it built and will try to build it all again, which fails with "already exists" errors. Never delete it. This is why the book includes backup AWS CLI scripts: they find resources by tag, not by state.

3.5 How references create order

You never tell Terraform what to build first. When resource B mentions `aws_vpc.this.id`, Terraform knows the VPC must exist before B. It draws a dependency graph and builds in the right order, in parallel where it can. Modules do not change this: the graph is per resource, not per folder.

4. The AWS Pieces, One at a Time

4.1 Networking

Piece	Analogy	What it really is
VPC	The fenced campus	A private virtual network with an IP range like 10.0.0.0/16
Subnet	A hallway	A smaller IP range, always in exactly one Availability Zone
Availability Zone	A separate building with its own power	A physically separate data center inside a region
Route table	The map on the wall	Rules that say where packets may go. Ours has only the built-in local route
Internet gateway	The front gate	Not built in this project
VPC endpoint (gateway)	Free delivery door to the package warehouse	A route-table entry pointing at S3; no network card; no hourly cost
VPC endpoint (interface)	Delivery door with a doorman (costs money)	A network card in your subnet with a private IP for one AWS service
Security group	Hall monitor at a door	A firewall attached to a resource: which ports, from whom

CIDR in one minute

10.0.0.0/16 means every address that starts with 10.0. (65,536 of them). 10.0.1.0/24 means every address that starts with 10.0.1. (256). The bigger the number after the slash, the smaller the group.

ANALOGY: Zip codes

A /16 is like a whole city zip-code prefix. A /24 is one street. Subnets are streets carved out of the city.

4.2 Compute

Piece	Analogy	What it really is
EC2 instance	A rented computer	A virtual server
AMI	The starting hard drive	The operating-system image a server boots from. We use Amazon Linux 2023
Launch template	Substitute-teacher packet	AMI, size, security group, badge and first-boot script, saved as a reusable recipe
User data	First-day to-do list	A script that runs once, the first time a server boots
Auto Scaling Group	The vice-principal	Keeps N servers running from the template, replaces broken ones
IAM role	Staff badge	Permissions a server carries; no passwords on disk
Instance profile	The lanyard	The thing that clips a role onto an EC2 instance
Session Manager	The intercom	A shell or port-forward into a server with no SSH and no public IP

4.3 Data

Piece	Analogy	What it really is
RDS	The records office and librarian	A managed PostgreSQL database; AWS patches and backs it up
DB subnet group	The two hallways the office may use	RDS requires subnets in at least two AZs
Snapshot	A photocopy of every file	A point-in-time backup you can restore as a new database
Secrets Manager	The principal's safe	Stores passwords; badges decide who may open it

4.4 Identity (stacks 5 and 6)

Piece	Analogy	What it really is
Keycloak	The front office that checks student IDs	A login server that apps trust instead of keeping their own passwords
Realm	One school inside a district	A self-contained set of users, clients and rules inside Keycloak
OIDC client	An app's visitor pass	A registered app allowed to send people to Keycloak to log in
OIDC	The hall-pass procedure	The standard steps for an app to hand a person to a login server and get a signed note back
NiFi	The mail room that sorts and forwards	A drag-and-drop tool for moving and transforming data

5. Step by Step: Build Everything

5.1 What you need on your laptop

Tool	Why	Check
AWS account and access keys	Somewhere to build	<code>aws sts get-caller-identity</code>
AWS CLI v2	Your laptop talks to AWS	<code>aws --version</code>
Session Manager plugin	Reach servers with no public IP	<code>session-manager-plugin --version</code>
Terraform 1.5 or newer	The builder	<code>terraform version</code>

Run `aws configure` once and paste your access key, secret key, region (us-east-1) and output format (json).

5.2 Stack 1: the network

```
bash
cd 01-network
terraform init
terraform plan
terraform apply      # type yes; about 3 minutes
cd ..
```

Outputs: `vpc_id`, `app_subnet_ids`, `db_subnet_ids`. Nothing downstream reads these outputs; later stacks look things up by tag.

5.3 Stack 2: the database

```
bash
cd 02-database
terraform init
terraform plan      # watch the data lookups find demo-vpc and the db subnets
terraform apply      # 8 to 12 minutes; RDS is slow
cd ..
```

Check the secret landed in the safe:

```
bash
aws secretsmanager get-secret-value --secret-id demo/db-credentials \
  --query SecretString --output text
# {"username":"rcadmin","password":"changeme","dbname":"appdb","port":5432}
```

5.4 Stack 3: the servers and the website

```
bash
cd 03-compute
terraform init
```

```
terraform plan
terraform apply      # about 2 minutes, then wait 3 to 4 more for first boot
```

5.5 Use the website through the intercom

There is no public IP, so you open a private tunnel with Session Manager port forwarding. The compute stack prints the exact command:

```
bash
cd 03-compute
eval "$(terraform output -raw port_forward_command)"
# Waiting for connections... (leave this terminal open)
```

Now open **http://localhost:8080** in your browser. Add a note, edit it, delete it. Each click is a Create, Read, Update or Delete against PostgreSQL.

5.6 Prove the rows are really in the database

```
bash
aws ssm start-session --target $(cd 03-compute && terraform output -raw ec2_instance_id)
# now on the server:
sudo dnf install -y postgresql7
S=$(aws secretsmanager get-secret-value --secret-id demo/db-credentials --region us-east-1
--query SecretString --output text)
export PGPASSWORD=$(echo "$S" | python3 -c 'import sys,json;print(json.load(sys.stdin)
["password"])')
psql -h $(grep DB_HOST /opt/app/app.env | cut -d= -f2) -U radmin -d appdb -c 'SELECT *
FROM notes;'
```

5.7 Optional stack 4: restore from a snapshot

```
bash
aws rds create-db-snapshot --db-instance-identifier demo-db --db-snapshot-identifier demo-
db-snap-1
aws rds wait db-snapshot-available --db-snapshot-identifier demo-db-snap-1
cd 04-database-restore && terraform init && terraform apply
aws secretsmanager get-secret-value --secret-id demo/demo-db-restored-credentials --query
SecretString --output text
```

5.8 Optional stacks 5 and 6: Keycloak and NiFi

Stage the tarballs in S3 first (from your laptop, which has internet), then apply both stacks and open two tunnels:

```
bash
./scripts/upload-artifacts.sh my-artifacts-bucket 26.3.2 2.4.0
# paste the bucket/key lines it prints into 05-keycloak/terraform.tfvars and
```

```
06-nifi/terraform.tfvars
(cd 05-keycloak && terraform init && terraform apply)      # wait ~3 min after apply
(cd 06-nifi      && terraform init && terraform apply)      # wait ~4 min after apply
# terminal A
(cd 05-keycloak && eval "$(terraform output -raw port_forward_command)")
# terminal B
(cd 06-nifi && eval "$(terraform output -raw port_forward_command)")
```

Open <https://localhost:8443/nifi>, accept the certificate warning, and you are sent to Keycloak at <http://localhost:8080>. Log in as nifiadmin with `nifi_user_password` from the `demo/keycloak-credentials` secret. You land back in NiFi as an admin.

5.9 Tear it all down

Reverse order, always. AWS refuses to delete a VPC that still has things inside it.

```
bash
(cd 06-nifi      && terraform destroy)
(cd 05-keycloak  && terraform destroy)
(cd 04-database-restore && terraform destroy)
(cd 03-compute   && terraform destroy)
(cd 02-database  && terraform destroy)
(cd 01-network   && terraform destroy)
```

GOTCHA: Snapshots and the keycloak database survive destroy

Manual RDS snapshots you created with the CLI and the keycloak database Keycloak created inside RDS are not tracked by Terraform. Delete the snapshot with `aws rds delete-db-snapshot`; the keycloak database disappears when the RDS instance is destroyed.

Quiz A

1. In which order do you apply the stacks? Destroy them?
2. Why can you not open the website with the server's IP from your laptop?
3. What does `terraform plan` do in Stack 2 before planning anything new?
4. What is the difference between `resource` and `data`?

6. Line by Line: Files Every Stack Shares

6.1 versions.tf

```
hcl
terraform {
  required_version = ">= 1.5.0"
```

What: refuse to run on Terraform older than 1.5. **Why:** older versions do not understand some syntax used here. Pinning avoids mystery errors on a teammate's laptop.

```
hcl
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

What: download the official AWS plugin, any 5.x version. **Why:** `~> 5.0` allows 5.1, 5.60, but not 6.0. Major versions change behavior; you want to upgrade on purpose, not by accident.

```
hcl
provider "aws" {
  region = var.region

  default_tags {
    tags = {
      Project    = var.project_name
      ManagedBy = "terraform"
    }
  }
}
```

What: configure the plugin for one region and stamp two tags on every resource. **Why:** tags are how humans, the billing console, and our backup CLI scripts find things. `ManagedBy = terraform` tells the next admin "do not edit this in the console; change the code".

ANALOGY: Name tags on lunch boxes

In a classroom every lunch box gets a name label so it can be found and returned. `default_tags` writes the label on everything automatically so nobody forgets.

6.2 variables.tf and terraform.tfvars

```
hcl
variable "vpc_cidr" {
  description = "IP range for the whole VPC"
```

```
type      = string
default   = "10.0.0.0/16"
}
```

Every knob has a description (for humans), a type (so a typo like a number where text is expected fails early) and a default (so the code works with no tfvars at all). `terraform.tfvars` holds `name = value` lines; anything there overrides the default.

KEY TIP: Defaults plus tfvars

Keep sensible defaults in `variables.tf` so a new teammate can run `apply` immediately. Keep environment-specific values (region, names, sizes) in `terraform.tfvars`. Never put passwords in either; use Secrets Manager.

7. Line by Line: Stack 1, Network

7.1 01-network/main.tf

```
hcl
module "vpc" {
  source = "../modules/vpc"

  name           = var.project_name
  region         = var.region
  vpc_cidr       = var.vpc_cidr
  azs            = var.azs
  app_subnet_cidrs = var.app_subnet_cidrs
  db_subnet_cidrs  = var.db_subnet_cidrs
  interface_endpoints = var.interface_endpoints
}
```

What: use the brick in `../modules/vpc` and hand it our settings. **Why** `../`: the modules folder sits one level up so all six stacks can share the same bricks.

7.2 modules/vpc/main.tf

```
hcl
resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = { Name = "${var.name}-vpc" }
}
```

What: the campus, 10.0.0.0/16. **Why DNS on:** RDS hands out a hostname, and the interface endpoints use private DNS so `secretsmanager.us-east-1.amazonaws.com` resolves to the delivery door instead of the internet. **The Name tag** becomes `demo-vpc`; Stacks 2 to 6 look the VPC up by that exact string.

```
hcl
resource "aws_subnet" "app" {
  count = length(var.app_subnet_cidrs)

  vpc_id           = aws_vpc.this.id
  cidr_block       = var.app_subnet_cidrs[count.index]
  availability_zone = var.azs[count.index]

  tags = {
    Name = "${var.name}-app-${count.index + 1}"
    Tier = "app"
  }
}
```

What: one app hallway per CIDR in the list. **How count works:** the list has 2 items, so Terraform makes `app[0]` and `app[1]`. `count.index` is 0 then 1, so subnet 0 uses 10.0.1.0/24 in us-east-1a and subnet 1 uses 10.0.2.0/24 in us-east-1b. That is the whole two-AZ trick. **Why the Tier tag:** later stacks search for `Tier = app`.

GOTCHA: Lists must match in length

If `app_subnet_cidrs` has three entries and `azs` has two, `var.azs[2]` does not exist and apply fails. Keep the lists the same length.

```
hcl
resource "aws_subnet" "db" {
  count = length(var.db_subnet_cidrs)
  ...
  tags = { Name = "${var.name}-db-${count.index + 1}", Tier = "db" }
}
```

Same pattern for the records-office hallways, tagged `Tier = db`. There is no `map_public_ip_on_launch` anywhere: nothing on this campus gets a public address.

```
hcl
resource "aws_route_table" "internal" {
  vpc_id = aws_vpc.this.id

  tags = { Name = "${var.name}-internal-rt" }
}
```

What: a route table with no route blocks. **Why that is enough:** AWS always adds one built-in route called `local` (10.0.0.0/16 stays inside). With no `0.0.0.0/0` route and no gateway, a packet addressed to the internet has nowhere to go. That single missing line is what "manual only within the VPC" means.

```
hcl
resource "aws_route_table_association" "app" {
  count = length(aws_subnet.app)

  subnet_id      = aws_subnet.app[count.index].id
  route_table_id = aws_route_table.internal.id
}
```

Hang the same wall map in every app hallway; the `db` association does the same for db hallways. Without an association a subnet silently uses the VPC's default route table, which is a common source of "why can't it reach X" confusion.

```
hcl
resource "aws_security_group" "endpoints" {
  name      = "${var.name}-vpce-sg"
  vpc_id    = aws_vpc.this.id

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
  }
}
```

```

    cidr_blocks = [var.vpc_cidr]
  }
  egress { from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"] }
}

```

What: the doorman at each delivery door: allow HTTPS (443) from anyone inside the campus. **Why 443 only:** AWS APIs speak HTTPS. **ingress** is incoming, **egress** is outgoing; **-1** means all protocols.

```

hcl
resource "aws_vpc_endpoint" "s3" {
  vpc_id          = aws_vpc.this.id
  service_name     = "com.amazonaws.${var.region}.s3"
  vpc_endpoint_type = "Gateway"
  route_table_ids = [aws_route_table.internal.id]
}

```

What: the free delivery door to the S3 warehouse. **How:** it adds a special route to our route table. **Why it matters:** Amazon Linux keeps its package repositories in S3, so this one block is what makes `dnf install` work with no internet. It is also how Keycloak and NiFi tarballs arrive in stacks 5 and 6.

```

hcl
resource "aws_vpc_endpoint" "interface" {
  for_each = toset(var.interface_endpoints)

  vpc_id          = aws_vpc.this.id
  service_name     = "com.amazonaws.${var.region}.${each.key}"
  vpc_endpoint_type = "Interface"
  subnet_ids      = aws_subnet.app[*].id
  security_group_ids = [aws_security_group.endpoints.id]
  private_dns_enabled = true
}

```

What: one doorman-guarded door per service in the list: `secretsmanager`, `ssm`, `ssmmessages`, `ec2messages`. **for_each versus count:** `for_each` names copies by value (`interface["ssm"]`) instead of by number, so removing one from the list does not renumber the rest. **subnet_ids = app[*].id:** a network card in every app hallway, so each door is reachable from both buildings. **private_dns_enabled:** the normal AWS hostname now points at the door.

Why these four: the website reads its password from `secretsmanager`; the other three are the trio Session Manager needs so you can log in and port-forward.

GOTCHA: Interface endpoints are the biggest bill

Each interface endpoint costs about \$0.01 per hour per network card. Four endpoints in two hallways is eight cards, about \$58 per month. If you never need a shell, drop the three `ssm` entries from `interface_endpoints` and keep only `secretsmanager`.

KEY TIP: Endpoints instead of a NAT gateway

A NAT gateway would let servers reach the whole internet for about \$32 per month plus data. Endpoints let servers reach only the AWS services you name. For a data platform that should never phone home, endpoints are

the safer default.

Quiz B

1. What route does the internal route table contain, and who added it?
2. Which endpoint makes `dnf install` work, and why is it free?
3. What does `for_each = toset([...])` do differently from `count`?
4. Why are the interface endpoints placed in both app subnets?

8. Line by Line: Stack 2, Database

8.1 02-database/data.tf

```
hcl
data "aws_vpc" "this" {
  filter {
    name   = "tag:Name"
    values = ["${var.project_name}-vpc"]
  }
}
```

What: find the VPC whose Name tag is demo-vpc. **Why a lookup and not a state file:** each stack stays independent. If Stack 1 has not been applied, this fails with a clear error, which is what you want.

```
hcl
data "aws_subnets" "db" {
  filter { name = "vpc-id"   values = [data.aws_vpc.this.id] }
  filter { name = "tag:Tier" values = ["db"] }
}
```

Every subnet in that VPC tagged `Tier = db`. Returns a list of IDs, in whatever order AWS gives them.

8.2 modules/rds/main.tf

```
hcl
resource "aws_secretsmanager_secret" "db" {
  name           = "${var.name}/db-credentials"
  recovery_window_in_days = 0
}
```

What: create the safe named demo/db-credentials. **Why recovery 0:** a deleted secret normally lingers 7 to 30 days in case you regret it; during that time you cannot recreate the same name. Zero lets destroy and re-apply work cleanly. For real data, use 7 or more.

```
hcl
resource "aws_secretsmanager_secret_version" "db" {
  secret_id = aws_secretsmanager_secret.db.id
  secret_string = jsonencode({
    username = var.db_username # radmin
    password = var.db_password # changeme
    dbname   = var.db_name
    port     = var.db_port
  })
}
```

What: put the contents in the safe as JSON. **Why JSON:** the website and Keycloak parse it by field name, so you can add fields later without breaking them. The `db_password` variable is marked `sensitive = true` so it is hidden in plan output; it is still in the state file, which is why state must be protected.

```
hcl
resource "aws_security_group" "app" {
  name     = "${var.name}-app-sg"
  vpc_id   = var.vpc_id

  ingress {
    from_port = var.app_port # 80
    to_port   = var.app_port
    protocol  = "tcp"
    cidr_blocks = [var.vpc_cidr]
  }
}
```

What: the hall-monitor list the **app servers** will wear. **Why it lives in the database stack:** the database's own monitor must point at this group, and Stack 2 is applied before Stack 3. Creating it here avoids a chicken-and-egg problem. It allows port 80 from anywhere inside the campus, which is how your port-forward tunnel arrives.

```
hcl
resource "aws_security_group" "db" {
  ingress {
    from_port = var.db_port # 5432
    to_port   = var.db_port
    protocol  = "tcp"
    security_groups = [aws_security_group.app.id]
  }
}
```

What: the librarian's list: PostgreSQL port 5432, **only** from things wearing the app security group. **Why not the VPC CIDR:** then any server on campus, including a compromised one, could reach the database. Group-to-group rules are the single most important security habit in this book.

ANALOGY: Badges, not addresses

The librarian does not ask "which hallway did you come from?" She asks "are you wearing the teacher badge?" That is a security-group reference. Keycloak in Stack 5 gets in simply by wearing the same badge.

```
hcl
resource "aws_db_subnet_group" "this" {
  name       = "${var.name}-db-subnets"
  subnet_ids = var.db_subnet_ids
}
```

RDS refuses to start unless you hand it a subnet group covering at least two AZs. That is the rule that makes our second db subnet required, not optional.

```
hcl
resource "aws_db_instance" "this" {
  identifier      = "${var.name}-db" # demo-db
  engine         = "postgres"
  engine_version = var.engine_version # 17
  instance_class = var.instance_class # db.t3.micro
}
```

```

allocated_storage = var.allocated_storage # 20 GB
db_name           = var.db_name           # appdb
username          = var.db_username       # rcadmin
password          = var.db_password
port              = var.db_port           # 5432

db_subnet_group_name = aws_db_subnet_group.this.name
vpc_security_group_ids = [aws_security_group.db.id]
publicly_accessible   = false
skip_final_snapshot   = true
}

```

What: the database. `engine_version = "17"` means newest 17.x. `publicly_accessible = false` means no public hostname. `skip_final_snapshot = true` is for practice only. **Later stacks look this up by the identifier `demo-db`.**

GOTCHA: `skip_final_snapshot` and real data

With `skip_final_snapshot = true`, terraform `destroy` deletes the database and every row in it, instantly and permanently. For anything real set it to `false`, add `final_snapshot_identifier`, and add `deletion_protection = true`.

GOTCHA: Reserved usernames

PostgreSQL on RDS rejects some master usernames such as `admin`, `postgres` and `rdsadmin`. That is why the tutorial uses `rcadmin`.

Quiz C

1. Why is the app security group created in the database stack?
2. Who may connect to port 5432?
3. What would you change before storing real data, and why?

9. Line by Line: Stack 3, Compute and the Website

9.1 03-compute/data.tf

```
hcl
data "aws_security_group" "app" {
  vpc_id = data.aws_vpc.this.id
  name   = "${var.project_name}-app-sg"
}
data "aws_db_instance" "this" {
  db_instance_identifier = "${var.project_name}-db"
}
data "aws_secretsmanager_secret" "db" {
  name = "${var.project_name}/db-credentials"
}
```

Three lookups of things Stack 2 built: the app badge (by name), the database (by identifier, giving us its address and port) and the safe (by name, giving us its ARN).

9.2 03-compute/main.tf

```
hcl
module "launch_template" {
  source              = "../modules/launch_template"
  security_group_id   = data.aws_security_group.app.id
  secret_arn          = data.aws_secretsmanager_secret.db.arn
  db_endpoint         = data.aws_db_instance.this.address
  db_port             = data.aws_db_instance.this.port
  db_name             = data.aws_db_instance.this.db_name
  user_data           = var.user_data
  user_data_file      = var.user_data_file
}
```

Hand the recipe everything the website needs.

```
hcl
module "asg" {
  subnet_ids          = data.aws_subnets.app.ids
  launch_template_id  = module.launch_template.launch_template_id
}
module "ec2" {
  name                = "${var.project_name}-app"
  subnet_id           = data.aws_subnets.app.ids[0]
  launch_template_id  = module.launch_template.launch_template_id
}
```

Both the group and the single server are stamped from the **same** recipe. Change the recipe once; both get it on their next replacement.

9.3 modules/launch_template/main.tf

```
hcl
data "aws_ssm_parameter" "al2023" {
  name = "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-default-x86_64"
}
```

What: look up the newest Amazon Linux 2023 image ID. **Why:** AMI IDs differ per region and go stale monthly. AWS publishes the current one in this public parameter, so the code never hard-codes an AMI.

```
hcl
resource "aws_iam_role" "instance" {
  name = "${var.name}-app-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
      Action    = "sts:AssumeRole"
    }]
  })
}
```

What: the staff badge and who may wear it: EC2 servers. `jsonencode` turns a Terraform map into the JSON text IAM wants, so you never fight with quotes.

```
hcl
resource "aws_iam_role_policy" "read_db_secret" {
  role = aws_iam_role.instance.id
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Action    = ["secretsmanager:GetSecretValue"]
      Resource  = var.secret_arn
    }]
  })
}
```

What: permission 1: open exactly one safe. **Why so narrow:** least privilege. If this server is ever compromised, the attacker gets one password, not every secret in the account.

```
hcl
resource "aws_iam_role_policy_attachment" "ssm" {
  role      = aws_iam_role.instance.name
  policy_arn = "arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"
}
```

What: permission 2, an AWS-managed policy that lets the Session Manager agent check in. Without it the ssm endpoints exist but the server never appears in `aws ssm describe-instance-information`.

```
hcl
```

```
resource "aws_iam_instance_profile" "instance" {
  role = aws_iam_role.instance.name
}
```

The lanyard. EC2 can only wear a role through an instance profile.

```
hcl
locals {
  user_data_vars = { secret_arn = var.secret_arn, db_endpoint = var.db_endpoint, ... }

  user_data = (
    var.user_data != "" ? var.user_data :
    var.user_data_file != "" ? templatefile(var.user_data_file, local.user_data_vars) :
    templatefile("${path.module}/user_data.sh.tpl", local.user_data_vars)
  )
}
```

What: choose the first-boot script. Two stacked if/else checks: inline text from tfvars wins, then a custom file, then the built-in one. `templatefile` reads a file and fills in `${...}` placeholders. `path.module` means "the folder this module lives in".

```
hcl
resource "aws_launch_template" "this" {
  image_id      = data.aws_ssm_parameter.al2023.value
  instance_type = var.instance_type
  vpc_security_group_ids = [var.security_group_id]
  iam_instance_profile { name = aws_iam_instance_profile.instance.name }
  user_data = base64encode(local.user_data)
}
```

The substitute-teacher packet: image, size, badge, hall-monitor list, first-day to-do list. `base64encode` is the format EC2 requires for user data.

GOTCHA: User data runs once

The first-boot script runs only the first time a server boots. Editing it does nothing to a running server. Run `terraform apply -replace=module.ec2.aws_instance.this` to get a fresh one; the ASG picks up the new template as it replaces servers.

9.4 modules/asg/main.tf

```
hcl
resource "aws_autoscaling_group" "this" {
  min_size      = var.min_size      # 1
  max_size      = var.max_size      # 2
  desired_capacity = var.desired_capacity # 1
  vpc_zone_identifier = var.subnet_ids
  health_check_type = "EC2"

  launch_template {
```

```

    id      = var.launch_template_id
    version = "$Latest"
  }
}

```

What: the vice-principal. Never fewer than 1 server, never more than 2, aim for 1 now, spread across both app hallways, replace any that stops. EC2 health check means "is it powered on and passing AWS status checks"; there is no load balancer to ask a smarter question.

KEY TIP: Scaling policies are separate

This ASG does not grow on its own; it only keeps the count you set. Add `aws_autoscaling_policy` resources (CPU target tracking is the simplest) when you want it to react to load.

9.5 modules/ec2/main.tf

```

hcl
resource "aws_instance" "this" {
  subnet_id = var.subnet_id

  launch_template {
    id      = var.launch_template_id
    version = "$Latest"
  }

  tags = { Name = var.name }
}

```

That is the entire module. The template already says what to build; this only says where. The Name tag is exactly what was passed in, which is how Stack 6 later finds `demo-keycloak`.

9.6 The website script, user_data.sh.tpl

The constraint shapes the script: **no internet**, so no `pip install`. Everything comes from the Amazon Linux repo (through the S3 door) or Python's standard library.

```

bash
dnf install -y python3 python3-psycopg2

```

Python and the PostgreSQL driver, both from the repo.

```

bash
cat > /opt/app/app.env <<'ENV'
SECRET_ARN=${secret_arn}
DB_HOST=${db_endpoint}
DB_PORT=${db_port}
DB_NAME=${db_name}
AWS_REGION=${region}
APP_PORT=${app_port}
ENV

```

A settings file. Terraform filled in the placeholders. Notice there is no password here, only the safe's address.

```
python
raw = subprocess.check_output([
    "aws", "secretsmanager", "get-secret-value",
    "--secret-id", os.environ["SECRET_ARN"], "--region", os.environ["AWS_REGION"],
    "--query", "SecretString", "--output", "text",
])
secret = json.loads(raw)
```

At startup the app shows its badge and asks Secrets Manager for the password using the AWS CLI that ships with Amazon Linux. It works because of the IAM policy and travels through the Secrets Manager door.

```
python
cur.execute("CREATE TABLE IF NOT EXISTS notes (id SERIAL PRIMARY KEY, text TEXT NOT NULL)")
```

Make the notes table once. `SERIAL` is PostgreSQL's auto-numbering.

```
python
cur.execute("SELECT id, text FROM notes ORDER BY id")           # READ
cur.execute("INSERT INTO notes (text) VALUES (%s)", (t,))      # CREATE
cur.execute("UPDATE notes SET text=%s WHERE id=%s", (t, nid))   # UPDATE
cur.execute("DELETE FROM notes WHERE id=%s", (nid,))            # DELETE
```

The four CRUD statements. %S placeholders let the driver escape user input so nobody can sneak SQL commands into a note. This is called preventing SQL injection.

ANALOGY: Order forms at the grocery deli

Handing the deli clerk a filled-in order form (placeholders) is safe. Letting a customer write anything on the clerk's own instruction sheet (string concatenation) is how someone writes "and give me the cash drawer". Always use the form.

```
python
class Handler(BaseHTTPRequestHandler):
    def do_GET(self): ... send_page() ...
    def do_POST(self):
        parts = self.path.strip("/").split("/")
        if parts[0] == "add": add_note(text)
        elif parts[0] == "update": update_note(int(parts[1]), text)
        elif parts[0] == "delete": delete_note(int(parts[1]))
        self.send_response(303); self.send_header("Location", "/")
```

A web server from the standard library. GET shows the page; POST reads the URL (`/add`, `/update/3`, `/delete/3`) to choose the CRUD function, then answers 303 which tells the browser to GET `/` again and see the fresh list.

```
ini
```

```
[Service]
EnvironmentFile=/opt/app/app.env
ExecStart=/usr/bin/python3 /opt/app/app.py
Restart=always
```

A systemd service so the app starts at boot and restarts if it crashes. `systemctl status app` and `journalctl -u app` are your first two troubleshooting commands.

Quiz D

1. Name the two permissions on the IAM role and what each is for.
2. Both the ASG and the EC2 use `version = "$Latest"`. What happens to a running server when you change the template?
3. Why does the script use the AWS CLI instead of `pip install boto3`?
4. What does `%S` protect against?

10. Line by Line: Stack 4, Restore from a Snapshot

ANALOGY: A photocopy of the filing cabinet

A snapshot is a photocopy of every file in the records office at one moment. Restoring it opens a second office with the same files. You change the lock on the new door (new password) so the old key does not work there.

10.1 modules/rds_from_snapshot/main.tf

```
hcl
data "aws_db_snapshot" "latest" {
  count          = var.snapshot_identifier == "" ? 1 : 0
  db_instance_identifier = var.source_db_identifier
  most_recent    = true
}
```

What: a lookup with `count` of 1 or 0: it only runs when you did **not** name an exact snapshot. `most_recent = true` picks the newest snapshot of the source database.

```
hcl
locals {
  snapshot_id = var.snapshot_identifier != "" ? var.snapshot_identifier :
data.aws_db_snapshot.latest[0].id
  secret_name = var.secret_name != "" ? var.secret_name : "${var.name}/${var.identifier}-
credentials"
}
```

Decide which snapshot and which secret name to use. `[0]` reaches into the counted data source.

```
hcl
resource "random_password" "db" {
  length      = var.password_length
  special     = true
  override_special = " !#$%^&*()-_+= "
}
```

What: the `random` provider invents a password. **Why `override_special`:** RDS rejects `/`, `@`, `"` and spaces in passwords. The value is stored in state so it does not change on every apply.

```
hcl
resource "aws_db_instance" "restored" {
  identifier          = var.identifier
  snapshot_identifier = local.snapshot_id
  instance_class      = var.instance_class
  password            = random_password.db.result
  db_subnet_group_name = aws_db_subnet_group.this.name
  vpc_security_group_ids = var.vpc_security_group_ids
  publicly_accessible = false
  skip_final_snapshot = var.skip_final_snapshot
}
```

The magic line is **snapshot_identifier**. Instead of an empty database, RDS restores this photocopy. Engine, version, **db_name**, storage size and the master **username** are locked to what the snapshot had. The **password** is the one thing you may reset on restore, so we hand it the new random one.

```
hcl
resource "aws_secretsmanager_secret_version" "db" {
  secret_string = jsonencode({
    username = aws_db_instance.restored.username
    password = random_password.db.result
    host     = aws_db_instance.restored.address
    port     = aws_db_instance.restored.port
    dbname   = aws_db_instance.restored.db_name
    engine   = aws_db_instance.restored.engine
  })
}
```

Write everything an app needs into the new safe. The username is read back from the restored instance because we never typed it; it came with the snapshot.

GOTCHA: Snapshots outlive Terraform

A snapshot you made with the CLI is not in any state file. `terraform destroy` will not delete it and it keeps costing storage. The destroy script removes manual snapshots for the project; otherwise use `aws rds delete-db-snapshot`.

KEY TIP: Use this for a test copy of production

Restoring last night's automated snapshot into a `-test` instance gives developers real-shaped data without touching production. Combine with a smaller `instance_class` to save money.

11. Line by Line: Stack 5, Keycloak

11.1 What Keycloak is doing here

ANALOGY: The front office checks IDs so teachers do not have to

Instead of every classroom (app) keeping a list of who is allowed in, students show their ID at the front office (Keycloak) and get a hall pass (a signed token). Each classroom trusts the hall pass. A realm is one school inside the district; a client is one classroom that has registered with the office.

Stack 5 uses only existing things: the VPC (by Name), the app subnets (by Tier), the RDS instance (by identifier), the RDS secret (by name) and the app security group (by name). All of those names are in `terraform.tfvars`, so the same code points at any environment.

11.2 modules/keycloak_launch_template/main.tf

```
hcl
resource "random_password" "admin"          { length = 20  special = false }
resource "random_password" "nifi_client_secret" { length = 32  special = false }
resource "random_password" "nifi_user"       { length = 16  special = false }
```

Three passwords invented once: the Keycloak admin, the OIDC client secret NiFi will use, and the test user's password.

```
hcl
resource "aws_secretsmanager_secret_version" "keycloak" {
  secret_string = jsonencode({
    admin_username = var.admin_username, admin_password = random_password.admin.result,
    realm = var.realm_name, nifi_client_id = var.nifi_client_id,
    nifi_client_secret = random_password.nifi_client_secret.result,
    nifi_user = var.nifi_user, nifi_user_password = random_password.nifi_user.result,
    nifi_user_email = var.nifi_user_email, public_url = var.public_url
  })
}
```

One safe holds everything. The Keycloak server reads it to configure itself; the NiFi server reads it for the client secret and admin email; you read it to log in.

```
hcl
vpc_security_group_ids = concat([aws_security_group.keycloak.id],
var.extra_security_group_ids)
```

Keycloak wears two badges: its own (port 8080 from the campus) **and** the existing `demo-app-sg`. Because the RDS monitor already trusts that badge, Keycloak reaches PostgreSQL with zero changes to the database stack.

```
hcl
Statement = concat(
  [{ Action = ["secretsmanager:GetSecretValue"],
    Resource = [var.db_secret_arn, aws_secretsmanager_secret.keycloak.arn] }],
```

```

var.artifact_bucket != "" ? [{ Action = ["s3:GetObject"],
    Resource = "arn:aws:s3:::${var.artifact_bucket}/*" }] : []
)

```

IAM: read the two safes and, only if a bucket was named, fetch from it. `concat` with a conditional empty list is the Terraform idiom for an optional policy statement.

11.3 modules/keycloak_launch_template/user_data.sh.tpl

```

bash
exec > /var/log/keycloak-setup.log 2>&1

```

Send every line of output to one log file. When something goes wrong on first boot, this file is the first thing you read.

```

bash
dnf install -y java-21-amazon-corretto-headless
dnf install -y postgresql17 || dnf install -y postgresql16

```

Keycloak 26 needs Java 21. `psql` is used once, to create a database; either client version is fine for that.

```

bash
if [ -n "${artifact_key}" ]; then
    aws s3 cp "s3://${artifact_bucket}/${artifact_key}" /tmp/keycloak.tar.gz --region $
{region}
else
    curl -fsSL -o /tmp/keycloak.tar.gz "${download_url}"
fi

```

Fetch the tarball from S3 through the free door when a key was given; otherwise a direct download (only works in a VPC that has internet).

```

bash
EXISTS=$(psql -h ${db_endpoint} -U "$DB_USER" -d ${db_admin_database} -tAc \
    "SELECT 1 FROM pg_database WHERE datname='${keycloak_db_name}')"
if [ "$EXISTS" != "1" ]; then
    psql ... -c "CREATE DATABASE ${keycloak_db_name}"
fi

```

Keycloak wants its own database inside the RDS instance. Create it only if missing so a rebuilt server reuses the existing one and keeps its users.

```

bash
cat > /opt/keycloak/data/import/${realm_name}-realm.json <<JSON
{ "realm": "nifi", "enabled": true,
  "clients": [ { "clientId": "nifi", "publicClient": false, "secret": "$NIFI_SECRET",
    "standardFlowEnabled": true,
    "redirectUris": ["https://localhost:8443/*", "https://*:8443/*"] } ],
  "users": [ { "username": "nifiadmin", "email": "nifiadmin@example.com",

```

```
"emailVerified": true,
  "credentials": [ { "type": "password", "value": "$NIFI_PASS" } ] } ] }
```

JSON

The sample realm as a file. `standardFlowEnabled` is the browser-redirect flow NiFi uses. `redirectUri` is the list of places Keycloak may send the user back to. `emailVerified` matters because NiFi identifies people by email.

```
ini
KC_DB=postgres
KC_DB_URL=jdbc:postgresql://${db_endpoint}:${db_port}/${keycloak_db_name}
KC_BOOTSTRAP_ADMIN_USERNAME=$ADMIN_USER
KC_BOOTSTRAP_ADMIN_PASSWORD=$ADMIN_PASS
KC_HTTP_ENABLED=true
KC_HOSTNAME=${public_url}
KC_HOSTNAME_STRICT=false
KC_HOSTNAME_BACKCHANNEL_DYNAMIC=true
```

Keycloak's settings. The last three solve the two-audiences problem: your **browser** reaches Keycloak as `localhost:8080` through the tunnel, while **NiFi's server** reaches it as `10.0.x.x:8080`. `KC_HOSTNAME` fixes the browser-facing URL and the token issuer; `BACKCHANNEL_DYNAMIC` lets server-to-server endpoints use whatever host was called.

```
ini
ExecStart=/opt/keycloak/bin/kc.sh start --import-realm
```

Start in production mode and import any realm files found in `data/import`. Realms that already exist are skipped, so restarts are safe.

GOTCHA: Plain HTTP is only acceptable inside the fence

Keycloak here speaks HTTP because nothing leaves the VPC. Anywhere a browser reaches it over a real network, use HTTPS, a real hostname in `KC_HOSTNAME`, and remove `KC_HTTP_ENABLED`.

12. Line by Line: Stack 6, NiFi with OIDC

12.1 06-nifi/data.tf

```
hcl
data "aws_instance" "keycloak" {
  count = var.keycloak_private_ip == "" ? 1 : 0
  filter { name = "tag:Name",          values = ["demo-keycloak"] }
  filter { name = "instance-state-name", values = ["running"] }
}
locals {
  keycloak_ip = var.keycloak_private_ip != "" ? var.keycloak_private_ip :
data.aws_instance.keycloak[0].private_ip
}
```

Find the running Keycloak server by its Name tag, unless tfvars gives an IP or hostname directly (for a Keycloak that lives elsewhere).

12.2 modules/nifi_launch_template/user_data.sh.tpl

```
bash
keytool -genkeypair -alias nifi -keyalg RSA -keysize 2048 -validity 825 \
  -keystore $CONF/keystore.p12 -storetype PKCS12 -storepass "$KS_PASS" \
  -dname "CN=nifi" -ext "SAN=dns:localhost,ip:127.0.0.1,ip:$PRIVATE_IP"
```

NiFi refuses to do OIDC over plain HTTP, so it makes itself a self-signed certificate valid for `localhost` (your tunnel) and its private IP. Your browser will warn once; that is expected.

```
bash
setp nifi.web.https.host 0.0.0.0
setp nifi.web.https.port ${https_port}
setp nifi.web.proxy.host "${proxy_hosts},${PRIVATE_IP}:${https_port}"
```

Listen on all interfaces. `proxy.host` is NiFi's list of names it accepts in the address bar; `localhost:8443` is what you use through the tunnel. Forget this and NiFi answers "invalid host header".

```
bash
setp nifi.security.user.login.identity.provider ""
setp nifi.security.user.authorizer managed-authorizer
```

Turn **off** NiFi 2's default single-user login and turn **on** the file-based authorizer that works with external identity.

```
bash
setp nifi.security.user.oidc.discovery.url
"http://${keycloak_host}:${keycloak_port}/realms/${realm_name}/.well-known/openid-configuration"
setp nifi.security.user.oidc.client.id "$CLIENT_ID"
setp nifi.security.user.oidc.client.secret "$CLIENT_SECRET"
setp nifi.security.user.oidc.claim.identifying.user email
setp nifi.security.user.oidc.additional.scopes email
```

The five OIDC lines. The discovery URL is where NiFi learns Keycloak's endpoints and public keys. `claim.identifying.user = email` means the NiFi identity is the email address.

bash

```
sed -i 's|<property name="Initial User Identity
1"></property>|...>nifiadmin@example.com<...|' authorizers.xml
sed -i 's|<property name="Initial Admin
Identity"></property>|...>nifiadmin@example.com<...|' authorizers.xml
```

Tell the authorizer that `nifiadmin@example.com` exists and is the first admin. This must match exactly the identity OIDC produces, which is why both servers read the same secret.

bash

```
/opt/nifi/bin/nifi.sh set-sensitive-properties-key "${sensitive_props_key}"
```

NiFi encrypts sensitive processor properties (like database passwords inside flows) with this key. It must be set before the first start.

GOTCHA: Identity strings must match character for character

OIDC gives NiFi `nifiadmin@example.com`. If `authorizers.xml` says `NifiAdmin@example.com`, NiFi treats them as two different people and you get "insufficient permissions". Same secret, same field, no retyping.

Quiz E

1. Which Keycloak setting lets the browser use localhost:8080 while NiFi's server uses the private IP?
2. How does the nifi realm get created without anyone clicking in the admin console?
3. Why does Keycloak need no change to the RDS security group?
4. Which claim does NiFi use to name the user, and where else must that exact value appear?

13. Line by Line: The AWS CLI Scripts

Terraform is the normal way to look at and remove resources. These scripts are the backup plan: they find things by the `Project` tag or by name, so they work even when a state file is gone.

13.1 scripts/aws-view.sh

```
bash
set -euo pipefail
P="${1:-demo}"; R="${2:-us-east-1}"; export AWS_DEFAULT_REGION="$R"
```

`set -e` stops on the first error, `-u` errors on unset variables, `pipefail` catches failures inside pipes. `${1:-demo}` means "first argument, or demo if none". Setting `AWS_DEFAULT_REGION` saves typing `--region` on every line.

```
bash
aws ec2 describe-vpcs --filters "Name=tag:Project,Values=$P" \
  --query 'Vpcs[].{Id:VpcId,Cidr:CidrBlock,Name:Tags[?Key==`Name`][0].Value}' --output
table
```

What: list VPCs tagged `Project=demo`. **The `--query` part** is JMESPath: pick a few fields and rename them. `Tags[?Key==\Name\][0].Value` means "from the tags, the one whose Key is Name, take its Value". `--output table` prints a readable grid.

```
bash
aws ec2 describe-vpc-endpoints --filters "Name=tag:Project,Values=$P" \
  --query 'VpcEndpoints[.
{Id:VpcEndpointId,Service:ServiceName,Type:VpcEndpointType,State:State}]' --output table
```

The delivery doors. A door in state `pendingAcceptance` or `failed` explains why a server cannot reach Secrets Manager.

```
bash
aws elbv2 describe-target-health --target-group-arn "$TG"
```

(Present in the earlier single-stack version.) Whether the load balancer thinks each server is healthy.

```
bash
aws rds describe-db-instances \
  --query "DBInstances[?contains(DBInstanceIdentifier, '$P')].
{Id:DBInstanceIdentifier,Engine:Engine,Ver:EngineVersion,Status:DBInstanceStatus,Endpoint:
Endpoint.Address}"
```

RDS has no tag filter on this call, so we filter by identifier containing the project name. `Status` must be `available` before apps can connect.

```
bash
aws ssm describe-instance-information --query 'InstanceInformationList[.
{Id:InstanceId,Ping:PingStatus}]'
```

Which servers have checked in with Session Manager. If your instance is missing here, port-forwarding will fail with `TargetNotConnected`: check the IAM policy and the three ssm endpoints.

13.2 scripts/aws-destroy.sh

bash

```
read -r -p "DELETE all '$P' resources in $R? Type 'yes': " OK
[ "$OK" = "yes" ] || { echo "Aborted."; exit 1; }
```

A deliberate speed bump. Destroying by CLI is irreversible.

bash

```
aws autoscaling update-auto-scaling-group --auto-scaling-group-name "$ASG" \
  --min-size 0 --max-size 0 --desired-capacity 0
aws autoscaling delete-auto-scaling-group --auto-scaling-group-name "$ASG" --force-delete
```

Why scale to zero first: the vice-principal would otherwise replace every server you terminate. Set the target to 0, then delete the group.

bash

```
aws ec2 terminate-instances --instance-ids $IDS
aws ec2 wait instance-terminated --instance-ids $IDS
```

Terminate the standalone servers and **wait**. Security groups cannot be deleted while a network card still uses them; waiting avoids a wall of `DependencyViolation` errors.

bash

```
aws iam remove-role-from-instance-profile ...
aws iam delete-instance-profile ...
aws iam delete-role-policy ... ; aws iam detach-role-policy ... ; aws iam delete-role ...
```

IAM cleanup has a strict order: unclip the badge from the lanyard, delete the lanyard, remove every policy from the badge, delete the badge.

bash

```
aws rds delete-db-instance --db-instance-identifier "$DB" --skip-final-snapshot --delete-automated-backups
aws rds wait db-instance-deleted --db-instance-identifier "$DB"
```

Delete the database without a final snapshot and wait; the subnet group cannot go until the instance is gone.

bash

```
aws secretsmanager delete-secret --secret-id "$S" --force-delete-without-recovery
aws rds delete-db-snapshot --db-snapshot-identifier "$SNAP"
```

Secrets and manual snapshots would otherwise linger (and bill) after everything else is gone.

bash

```
aws ec2 delete-vpc-endpoints --vpc-endpoint-ids "$EP"
sleep 60
for i in 1 2 3; do
  ... aws ec2 delete-security-group --group-id "$G" 2>/dev/null && echo "deleted $G"
  sleep 10
done
```

Delete the doors, then wait a full minute: their network cards release slowly. Then try deleting security groups three times with pauses, ignoring failures, because the order they free up is unpredictable.

```
bash
aws ec2 delete-subnet ...; aws ec2 delete-route-table ...; aws ec2 delete-vpc --vpc-id
"$VPC"
```

Hallways, wall map, campus. The route-table query skips the main table (`Associations[0].Main != true`) because AWS deletes that one with the VPC.

13.3 scripts/upload-artifacts.sh

```
bash
aws s3api head-bucket --bucket "$B" 2>/dev/null || aws s3 mb "s3://$B" --region "$R"
curl -fL -o "keycloak-$KC.tar.gz"
"https://github.com/keycloak/keycloak/releases/download/$KC/keycloak-$KC.tar.gz"
curl -fL -o "nifi-$NF-bin.zip" "https://archive.apache.org/dist/nifi/$NF/nifi-$NF-
bin.zip"
aws s3 cp "keycloak-$KC.tar.gz" "s3://$B/keycloak-$KC.tar.gz"
aws s3 cp "nifi-$NF-bin.zip" "s3://$B/nifi-$NF-bin.zip"
```

Run from a laptop **with** internet. Create the bucket if it does not exist, download both tarballs, upload them. `archive.apache.org` keeps every NiFi version; the main download mirror deletes old ones. It then prints the exact tfvars lines to paste.

ANALOGY: Stocking the school store before opening day

The campus has no gate, so the store cannot order from suppliers. The night before, you drive to the warehouse yourself and leave the boxes at the delivery door. That is what this script does with S3.

Quiz F

1. Why does the destroy script scale the ASG to zero before deleting it?
2. Why must VPC endpoints be deleted before security groups?
3. What does `--query 'Vpcs[ ].{Id:VpcId}'` do?

14. Best Practices

14.1 Terraform habits

Habit	Why	In this project
Modules for every repeatable thing	One fix improves every copy	<code>modules/</code> shared by six stacks
Defaults in <code>variables.tf</code> , values in <code>tfvars</code>	Works out of the box, easy to point elsewhere	Every stack
Pin Terraform and provider versions	No surprise behavior changes	<code>versions.tf</code>
<code>default_tags</code> on the provider	Everything is findable and billable	<code>Project</code> , <code>ManagedBy</code>
Never hard-code IDs	AMIs and subnet IDs change per region and per rebuild	SSM AMI lookup, tag lookups
Separate stacks by rate of change and owner	Small blast radius	01 network / 02 data / 03 app
Look up cross-stack resources by tag or name	No shared state, no ordering fragility	<code>data.aws_vpc</code> , <code>data.aws_subnets</code>
<code>terraform fmt</code> and <code>validate</code> before every commit	Catches typos in seconds	Do this locally or in CI
Read every plan; <code>grep</code> for <code>destroy</code>	Terraform only warns once	Habit
Remote state with locking (S3 + DynamoDB)	Two people cannot corrupt state at once	Not set up here; add before a team uses it

14.2 Security habits

- **Security-group-to-security-group rules.** Never open a database to a CIDR when a group reference will do.
- **No passwords on disk, in user data, or in `tfvars`.** Servers read Secrets Manager with an IAM role. The only exception in this book is the tutorial password `changeme`, which you should change immediately for real work.
- **Least-privilege IAM.** One secret ARN, one bucket, nothing else. Add permissions when a log tells you to, not before.
- **No public IPs, no SSH.** Session Manager gives a shell and a tunnel with full audit logging and no open port 22.
- **Endpoints instead of a NAT gateway** when servers only need AWS services.
- **Rotate the RDS password** and let RDS manage it: `manage_master_user_password = true` generates and rotates it in Secrets Manager for you.

14.3 Data and database habits

- `skip_final_snapshot = false`, `final_snapshot_identifier`, and `deletion_protection = true` for anything you would cry about losing.
- `backup_retention_period` of 7 or more days so point-in-time restore works.
- `multi_az = true` for production; it doubles the cost and removes the single point of failure.
- Give each application its **own database role** with rights only on its own database. Keycloak in Stack 5 borrows the master user for simplicity; a real setup would create a `keycloak` role.

- Test restores. A backup you have never restored is a hope, not a backup. Stack 4 is that test.

14.4 Operations habits

- Send first-boot output to a log file (`exec > /var/log/x-setup.log 2>&1`) so the first question, "what happened on boot?", has an answer.
- Run everything as a systemd service with `Restart=always`.
- Tag with an `Owner` and `Environment` in addition to `Project` once more than one person is involved.
- Set a billing alarm before you build anything.

15. Gotchas: The Mistakes Everyone Makes Once

Gotcha	What you see	Fix
Applying stacks out of order	no matching VPC found	01, then 02, then 03; destroy in reverse
Destroying out of order	DependencyViolation on the VPC	Destroy 03 before 02 before 01
Lost terraform.tfstate	Plan wants to create things that exist	Never delete it; use <code>scripts/aws-destroy.sh</code> to clean up, then start over
Edited user data, nothing changed	Old website still running	User data runs once; <code>terraform apply -replace=...</code>
Interface endpoint bill	\$58/month you did not expect	Remove the ssm entries or destroy Stack 1 between sessions
TargetNotConnected on port-forward	Cannot reach the server	Wait 2 minutes; check the IAM SSM policy and the three ssm endpoints; install the Session Manager plugin
Security group cannot be deleted	DependencyViolation	A network card (endpoint, instance, RDS) still uses it; wait and retry
Secret name already exists	ResourceExistsException on re-apply	A deleted secret sits in recovery; use <code>recovery_window_in_days = 0</code> for demos or <code>--force-delete-without-recovery</code>
Reserved RDS username	MasterUsername admin cannot be used	Pick another, e.g. rcadmin
Password with / or @	RDS rejects it	<code>override_special</code> in <code>random_password</code>
<code>\${</code> in a template file	templatefile error: unknown variable	Bash <code>\${VAR}</code> collides with Terraform; write <code>\$VAR</code> or escape as <code>`\${VAR}</code>
Lists of different lengths	index out of range	azs and subnet CIDR lists must match
NiFi invalid host header	Blank page or error	Add the host you typed to <code>nifi.web.proxy.host</code>
NiFi insufficient permissions after login	Logged in but locked out	Initial Admin Identity must equal the OIDC email exactly
Keycloak redirect goes to the private IP	Browser cannot connect	<code>KC_HOSTNAME</code> must be the URL your browser uses; keep <code>BACKCHANNEL_DYNAMIC=true</code>
Tarball download fails on the server	curl timeout	There is no internet; stage the file in S3 and set <code>artifact_key</code>
Snapshot left behind	Small ongoing charge	<code>aws rds delete-db-snapshot</code>

16. Key Tips

KEY TIP: Read the plan like a bank statement

Count the adds, changes and destroys. If any number surprises you, stop.

KEY TIP: One terminal per tunnel

Session Manager port-forwards block the terminal. Keycloak in one, NiFi in another, the shell in a third.

KEY TIP: The three commands that solve most server problems

`sudo systemctl status <svc>`, `sudo journalctl -u <svc> -n 100`, and `cat /var/log/<svc>-setup.log`.

KEY TIP: Name things so they can be found

Every cross-stack lookup in this project is a Name tag, a Tier tag, an identifier or a secret name. Consistent naming is the glue between stacks.

KEY TIP: Stop, do not destroy, between sessions

`aws ec2 stop-instances` and `aws rds stop-db-instance` pause the meter on compute while keeping data. RDS auto-restarts after 7 days.

KEY TIP: Copy the outputs somewhere

`terraform output` in each stack gives you the tunnel commands, endpoints and secret names. Paste them into your runbook.

KEY TIP: Make the change in code, not the console

If you fix something in the AWS console, the next `terraform apply` will undo it. Fix the code, apply, done.

17. Supporting Data and ETL Apps as a Cloud Team Admin

Once the campus exists, the cloud team's job changes from building to hosting. Data teams will ask you to run pipelines that pull, clean and load data. Here is how the pieces in this book map to that job, and a runbook to follow.

ANALOGY: The school kitchen

An ETL pipeline is the school kitchen: ingredients arrive (Extract), get chopped and cooked (Transform), and land on trays for the cafeteria (Load). The cloud team runs the kitchen: the ovens (compute), the pantry (storage), the recipe cards (launch templates), the keys to the walk-in fridge (secrets) and the delivery door (endpoints). The cooks (data engineers) bring recipes; you keep the kitchen safe, stocked and inspected.

17.1 What each piece is for in a data platform

Piece	Role for data / ETL
App subnets + ASG	Where scheduled ETL workers run. Scale the group up for a nightly batch, down after
Launch template + user data	The standard worker image: runtime, drivers, agent, log shipping. One template per kind of job
RDS PostgreSQL	Landing zone and metadata store for pipelines; the CRUD app is a stand-in for any app that reads and writes it
Secrets Manager + IAM role	How every job gets database and API credentials without a password in a script
S3 gateway endpoint	Raw files, exports and job artifacts move through S3 for free inside the region
Interface endpoints	Add logs, monitoring, sts, kms and glue endpoints as pipelines start needing them
Snapshots (Stack 4)	Nightly copies; restore into a test DB for developers
Keycloak (Stack 5)	Single login for NiFi, dashboards and admin tools; one place to disable a departing user
NiFi (Stack 6)	Visual pipeline builder; a common ETL entry point for teams that do not write code

17.2 Runbook: onboarding a new ETL app

- Ask three questions:** what does it read, what does it write, when does it run? The answers decide subnets, security groups and whether it needs an ASG schedule.
- Create its database role.** `CREATE ROLE etl_sales LOGIN PASSWORD '...'; GRANT ... ON SCHEMA sales.` Never hand out the master user.
- Store that credential in Secrets Manager** under a predictable name: `<project>/<app>-credentials`.
- Copy the launch_template module pattern:** its own IAM role reading only its own secret, its own security group, its own user data. Reuse `demo-app-sg` only if it truly needs the same access.
- Add the security-group reference** to the RDS security group's `allowed_sg_ids`, apply Stack 2, and verify with `aws ec2 describe-security-groups`.
- Add endpoints the app needs** (for example logs for CloudWatch, glue or athena) to `interface_endpoints` and apply Stack 1.
- Give it a schedule** with `aws_autoscaling_schedule` (scale to N at 01:00, to 0 at 05:00) or an EventBridge rule.

8. **Write its runbook page:** how to see logs, how to rerun, who to call, what it costs.

17.3 Runbook: daily and weekly checks

When	Check	Command
Daily	Everything running and healthy	<code>./scripts/aws-view.sh demo</code>
Daily	Servers reachable by Session Manager	<code>aws ssm describe-instance-information</code>
Daily	RDS status, storage and connections	<code>aws rds describe-db-instances</code> , <code>CloudWatch FreeStorageSpace</code> , <code>DatabaseConnections</code>
Daily	Last night's jobs	<code>journalctl -u <job></code> on the worker, or the job's own log table in PostgreSQL
Weekly	Snapshots exist and are recent	<code>aws rds describe-db-snapshots --db-instance-identifier demo-db</code>
Weekly	Cost against budget	AWS Budgets, <code>COSTS.md</code>
Weekly	Terraform drift	<code>terraform plan</code> in each stack should say no changes
Monthly	Rotate credentials, prune old snapshots, patch by replacing servers	<code>terraform apply -replace=...</code>
Quarterly	Test a restore (Stack 4)	Restore, run the app against it, destroy

17.4 Runbook: troubleshooting playbook

Symptom	Likely cause	Where to look
App cannot connect to the database	Security group, wrong endpoint, DB not available	<code>aws-view.sh</code> security groups; <code>/opt/app/app.env</code> ; RDS status
<code>AccessDeniedException</code> reading a secret	IAM policy names the wrong ARN	The role's inline policy; the secret's ARN
<code>dnf install</code> hangs	S3 gateway endpoint missing or wrong route table	<code>describe-vpc-endpoints</code> ; route table has the S3 prefix-list route
Secrets Manager call times out	No secretsmanager interface endpoint, or its SG blocks 443	<code>describe-vpc-endpoints</code> ; endpoint SG allows 443 from the VPC
ASG keeps replacing servers	First-boot script fails and the health check sees a bad instance	The setup log on a surviving instance; <code>describe-auto-scaling-groups</code> activity history
Nightly job ran twice	Two ASG instances both ran the scheduler	Use one scheduler instance or a database lock; only workers scale
Storage filling up	Logs or temp files on the root disk	<code>df -h</code> ; ship logs to CloudWatch; bigger <code>block_device_mappings</code> in the template
Slow queries after a restore	Restored DB has cold cache and smaller instance class	Wait, or match the source instance class

17.5 Growing this design

- **Private DNS names** (Route 53 private hosted zone) so apps say `db.demo.internal` instead of copying endpoints.
- **A scheduler** (EventBridge or an orchestrator like Airflow on its own template) instead of cron on a worker.
- **Log and metric shipping** through `logs` and `monitoring` endpoints; alarms on job failure and RDS storage.
- **Container images** in ECR (via an `ecr` endpoint) so a worker template runs any job image; ECS or EKS when the number of jobs grows.
- **A CI pipeline** that runs `terraform fmt`, `validate` and `plan` on every change and applies from a shared, locked state.
- **An internal load balancer** in front of the ASG when a data service must be reachable by other apps, not just by tunnel.

18. Quiz Answer Key

Quiz A

1. Apply 01, 02, 03 (then 04, 05, 06 as wanted). Destroy in reverse.
2. The VPC has no internet gateway and the server has no public IP; there is no path from your laptop. Session Manager tunnels are the way in.
3. It runs the data lookups to find the VPC and db subnets that Stack 1 created.
4. `resource` creates something; `data` looks up something that already exists.

Quiz B

1. Only `local` (10.0.0.0/16). AWS adds it automatically.
2. The S3 gateway endpoint. Gateway endpoints are route-table entries with no hourly charge.
3. `for_each` names copies by value (`interface["ssm"]`); `count` numbers them. Removing an item with `for_each` does not renumber the others.
4. So each door has a network card in both Availability Zones; a server in either building reaches it locally.

Quiz C

1. The DB security group must reference the app SG, and Stack 2 is applied before Stack 3.
2. Only resources wearing the app security group.
3. `skip_final_snapshot = false` with a `final_snapshot_identifier`, `deletion_protection = true`, a real password, `backup_retention_period` of 7 or more, and `multi_az` for production.

Quiz D

1. An inline policy allowing `secretsmanager:GetSecretValue` on one ARN (read the password) and `AmazonSSMManagedInstanceCore` (Session Manager).
2. Nothing; it keeps running the old version until it is replaced.
3. There is no internet, so PyPI is unreachable; the CLI ships with Amazon Linux.
4. SQL injection: the driver escapes user input so it cannot be run as SQL.

Quiz E

1. `KC_HOSTNAME_BACKCHANNEL_DYNAMIC=true` together with `KC_HOSTNAME` set to the browser URL.
2. `kc.sh start --import-realm` imports the JSON file the first-boot script wrote to `data/import`.
3. Keycloak wears the existing `demo-app-sg`, which the RDS security group already trusts.
4. The `email` claim; it must also appear as NiFi's Initial Admin Identity and Initial User Identity 1.

Quiz F

1. Otherwise the ASG replaces every server you terminate; scaling to zero stops that.
2. Interface endpoints hold network cards that use the endpoint security group; AWS will not delete a security group still referenced by a network card.
3. `JMESPath`: from the list of VPCs, build objects with one field named `Id` holding each `VpcId`.

19. Glossary

Term	Meaning
AMI	Amazon Machine Image: the disk image a server boots from
ARN	Amazon Resource Name: the unique ID of anything in AWS
ASG	Auto Scaling Group: keeps a set number of servers running
AZ	Availability Zone: an isolated data center within a region
CIDR	An IP range written like 10.0.0.0/16
CRUD	Create, Read, Update, Delete: the four basic data operations
Data source	A Terraform block that reads an existing resource
Endpoint (VPC)	A private connection from a VPC to one AWS service
ETL	Extract, Transform, Load: moving and reshaping data
IAM	Identity and Access Management: who may do what in AWS
Instance profile	The wrapper that attaches an IAM role to an EC2 instance
JMESPath	The query language used by <code>-query</code> in the AWS CLI
Launch template	A saved recipe for creating EC2 instances
Least privilege	Give exactly the permissions needed and nothing more
OIDC	OpenID Connect: a standard for logging in through a separate identity server
Realm	A self-contained set of users and apps in Keycloak
Security group	A firewall attached to a resource
Session Manager	AWS service for shell access and port-forwarding without SSH
Snapshot	A point-in-time copy of a database
State	Terraform's record of what it has built
Subnet	A slice of a VPC's IP range in one AZ
systemd	The Linux service manager; <code>systemctl</code> controls it
tfvars	A file of variable values that override defaults
User data	A script that runs on an instance's first boot
VPC	Virtual Private Cloud: your private network in AWS